

✓ ASL Image Data Loader & Augmentation Setup

This notebook sets up a small subset of the ASL Alphabet dataset for transfer learning.

Steps:

- Download and use the ASL dataset using kagglehub
- Work with just 5 common letters in English: E, A, R, I, O
- Limit each class to under 100 images
- Prepare training and validation loaders

```
# Install kagglehub (first-time only)
!pip install -q kagglehub
```

```
import kagglehub
```

```
# Download ASL Alphabet dataset
path = kagglehub.dataset_download("grassknotted/asl-alphabet")
print("Downloaded dataset path:", path)
```

```
🔄 Downloading from https://www.kaggle.com/api/v1/datasets/download/grassknotted/asl-alphabet?dataset_version_number=1...
100% ██████████ 1.03G/1.03G [00:09<00:00, 121MB/s] Extracting files...
```

```
Downloaded dataset path: /root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1
```

```
# Original dataset path
original_data_path = "/root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train"
```

```
import os
print(os.listdir(original_data_path))
```

```
🔄 ['S', 'R', 'N', 'K', 'T', 'H', 'X', 'U', 'M', 'J', 'space', 'I', 'D', 'B', 'O', 'P', 'Q', 'Y', 'E', 'del', 'W', 'Z', 'nothin
```

Dataset Overview

The ASL Alphabet dataset includes folders for each letter (A–Z), plus a few extra labels.

For this assignment, we are using a subset of just 5 letters:

['E', 'A', 'R', 'I', 'O']

We'll take up to 80 images per class and split them into training and validation sets. This keeps the dataset small and easy to use for transfer learning.

✓ Prepare Subset with 5 Letters

We'll now create a smaller dataset using just the letters: **E, A, R, I, O**.

For each class:

- We'll randomly pick 80 images
- Split them into training (56) and validation (24)
- Save them in a new folder: `asl_subset/train/` and `asl_subset/val/`

```
import os, shutil, random
from pathlib import Path
```

```
# Target subset directory
subset_root = Path("asl_subset")
train_dir = subset_root / "train"
val_dir = subset_root / "val"
```

```
# Letters we are keeping
selected_letters = ['E', 'A', 'R', 'I', 'O']
images_per_class = 80
train_ratio = 0.7
```

```
# Create new directories
```

```

# Create new directories
for split in [train_dir, val_dir]:
    for letter in selected_letters:
        os.makedirs(split / letter, exist_ok=True)

# Copy images for each letter
for letter in selected_letters:
    letter_path = Path(original_data_path) / letter
    all_images = list(letter_path.glob("*.jpg")) + list(letter_path.glob("*.jpeg")) + list(letter_path.glob("*.png"))
# fix: ASL dataset uses .png
if len(all_images) < images_per_class:
    raise ValueError(f"Not enough images in class {letter}. Found {len(all_images)}.")

selected_images = random.sample(all_images, images_per_class)

# Split into train and val
train_cutoff = int(train_ratio * images_per_class)
train_images = selected_images[:train_cutoff]
val_images = selected_images[train_cutoff:]

# Copy files
for img_path in train_images:
    shutil.copy(img_path, train_dir / letter / img_path.name)
for img_path in val_images:
    shutil.copy(img_path, val_dir / letter / img_path.name)

print("Subset created successfully at:", subset_root)

```

Subset created successfully at: asl_subset

✓ Load and Visualize Subset

Using Keras ImageDataGenerator to load the subset and visualize a few sample images. to confirm that:

- Images are being read correctly
- Labels match the folder names
- Class distribution looks good

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt

```

```

# Define image shape and batch
IMG_SIZE = (100, 100)
BATCH_SIZE = 16

```

```

# Data generators (no augmentation for now)
datagen = ImageDataGenerator(rescale=1./255)

```

```

train_generator = datagen.flow_from_directory(
    directory="asl_subset/train",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True
)

```

```

val_generator = datagen.flow_from_directory(
    directory="asl_subset/val",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True
)

```

Found 280 images belonging to 5 classes.
Found 120 images belonging to 5 classes.

```

# Visualize 1 batch of training images
images, labels = next(train_generator)

```

```

# Create index-to-label mapping
idx_to_class = {v: k for k, v in train_generator.class_indices.items()}

```

```

plt.figure(figsize=(10, 8))

```

```

plt.figure(figsize=(10, 8))
for i in range(12):
    plt.subplot(3, 4, i + 1)
    plt.imshow(images[i])
    label_idx = labels[i].argmax()
    plt.title(f"Label: {idx_to_class[label_idx]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

```

